CS559 · COMPUTER SYSTEMS DESIGN · LAB REPORT

Containerising the Proximity Search API

The Lab 7 service as a two-container Docker Compose stack, started with one command

AUTHOR
Kunal Sewal

STUDENT ID
12341270

DATE
27 October 2026

MACHINE (PERSONAL)
KS_Predator · Windows 11 Home

DOCKER
Desktop 4.54.0 · Engine 29.1.2 · Compose v2.40.3

SOURCE CODE
github.com/KunalSewal/Proximity-Search-API

ABSTRACT

The Proximity Search API from Lab 7 (Flask + gunicorn, Dijkstra over a road-linkage file) was packaged as two containers: an **api** service built from a multi-stage `python:3.12-slim` Dockerfile, and a **gateway** service built from `nginx:1.28-alpine` that acts as the reverse proxy and serves the link files over HTTP. A single `docker-compose.yml` builds both images, creates a private bridge network and a named log volume, and starts the API first and the gateway only once the API passes its health check. `docker compose up -d --build` is the only command needed. All work was done on my personal laptop (Docker Desktop on WSL 2). The test suite passes inside a container (23/23), and 200 random queries sent through the stack returned exactly the same top-10 lists as the engine run natively. Twelve problems came up along the way; each one, with its cause and fix, is recorded in Section 4.

1 command
`docker compose up -d --build` builds and starts the whole stack

2 services
api (gunicorn) behind gateway (nginx); only port 8080 is published

200/200
identical top-10 results, container vs. native engine (median 10.8 ms round trip)

7.4 s
cold start of the full stack from built images, including health checks

1 Architecture of the stack
2 Docker design choices
3 Steps followed, with screenshots
4 Errors faced and how they were resolved
5 Docker commands used
6 What I learnt
A Appendix: complete Docker files

1 Architecture of the stack

The Lab 7 application is one Python process. It loads the 10,000-location CSV at start-up, and for each `/search/` request it parses the link file (or reuses a cached graph) and runs Dijkstra. I split the stack into two services: the application itself, and an nginx gateway in front of it. The gateway is the only container that can be reached from the host. It enforces the upload limit, forwards requests to gunicorn, and serves the `links/` folder at `/files/`. That last job lets the API's "link as a URL" input work entirely inside the Docker network: `link=http://gateway/files/link.txt` is resolved by Docker's DNS to the gateway container.

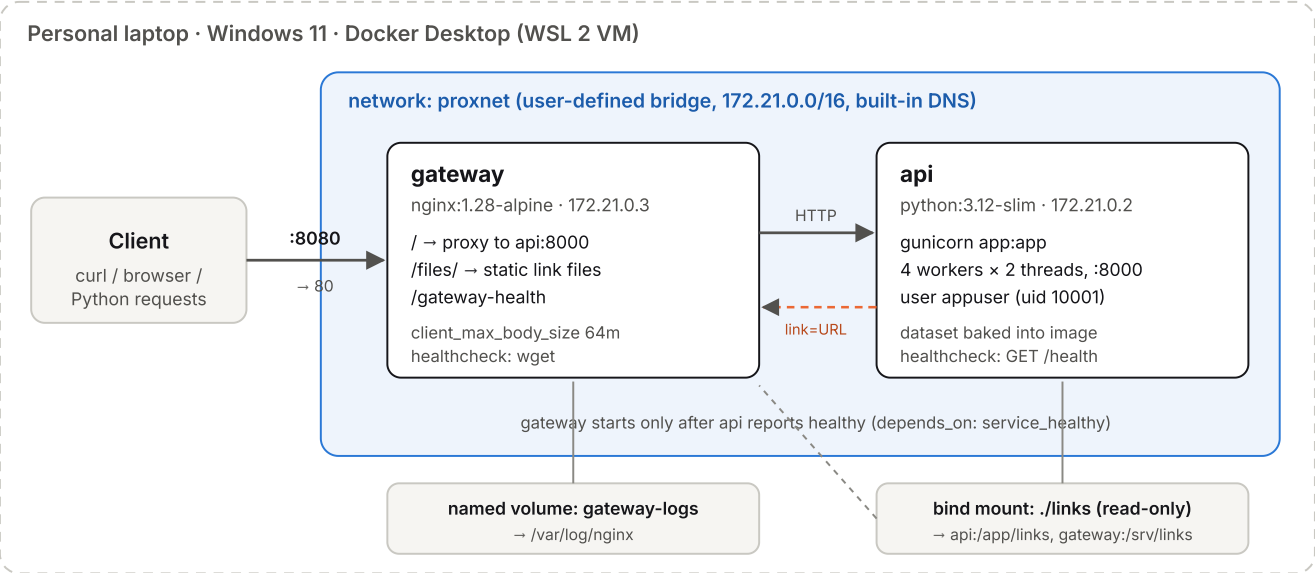


Figure 1. The stack. Only the gateway publishes a port, so the API can be reached only through nginx. The orange dashed arrow is the API fetching a link file from the gateway by URL. Both containers mount the same host `links/` folder read-only.

Files added to the repository

FILE	PURPOSE
`Dockerfile`	API image. Stages: `builder` (venv + dependencies) → `runtime`; `testdeps` (+ pytest) → `test`
`gateway/Dockerfile`	Gateway image: nginx with a config template and a health check
`gateway/default.conf.template`	nginx server block; `${API_HOST}`, `${API_PORT}`, `${MAX_UPLOAD}` are filled in at start-up
`docker-compose.yml`	Both services, the `proxnet` network, the `gateway-logs` volume, ports and environment
`gunicorn.conf.py`	gunicorn settings read from `PORT`, `WORKERS`, `THREADS`, so `CMD` can be exec form
`.dockerignore`	Keeps `.git`, the report, tools and stray copies of the dataset out of the build context
`.env.example`	Host-side settings (`HOST_PORT`, `API_WORKERS`, ...); copied to `.env`, which is git-ignored

2 Docker design choices

2.1 Base images

SERVICE	BASE IMAGE	WHY
api	`python:3.12-slim` (Debian, both stages)	Matches the Python 3.12 used in development. *slim* leaves out compilers and docs but keeps glibc, so wheels from PyPI install as-is (Alpine's musl would force source builds for some packages). Final image: 50 MB compressed content, 210 MB on disk.
gateway	`nginx:1.28-alpine`	The current stable nginx line, on Alpine because nothing here is compiled. The official image already runs `envsubst` over `/etc/nginx/templates/*.template`, so the upstream host and upload limit can come from environment variables without a custom entrypoint. Image: 26 MB compressed, 92.5 MB on disk.

Multi-stage build. The `builder` stage creates `/opt/venv` and installs Flask and gunicorn. The `runtime` stage starts again from a clean `python:3.12-slim` and copies in only the venv and the app (`app.py`, `engine.py`, `gunicorn.conf.py`, `data/`, `links/`), so pip's caches and build leftovers never reach the final image. `requirements.txt` is copied and installed before any source code, so editing `engine.py` reuses the cached dependency layer instead of reinstalling. A separate `testdeps` stage adds pytest to the same venv; the `test` target is `runtime` plus that venv plus `tests/`, so the shipped image never contains pytest.

Security. The API runs as `appuser` (uid 10001, no home directory, `nologin` shell). The application files are owned by root and read-only for that user, and both health checks use tools already in the base images (Python's `urllib`, BusyBox `wget`), so no curl package is added.

2.2 Ports

CONTAINER	PORT	EXPOSURE
gateway	`80/tcp`	Published as `${HOST_PORT:-8080}:80`, the only entry point from the host. 8080 avoids needing admin rights for port 80 on Windows; it can be changed in `.env` without editing YAML.
api	`8000/tcp`	`expose` only: reachable as `api:8000` on `proxnet`, not from the host (verified in Figure 9: `curl localhost:8000` fails).

2.3 Networks

One user-defined bridge network, `proxnet`. Compose would create a default network anyway, but naming it makes the topology explicit in the YAML and in `docker network inspect`. On a user-defined bridge Docker runs an embedded DNS server (`127.0.0.11`), so containers find each other by service name. nginx's `upstream` points at `api:8000`, and the API can download `http://gateway/files/link.txt`. I considered marking the network `internal: true` to cut the API off from the internet entirely, but that would break the API's documented feature of fetching a link file from any `http(s)` URL, so the network stays a normal bridge and isolation comes from not publishing the API's port.

2.4 Volumes

MOUNT	TYPE	REASON
`./links → /app/links:ro` (api) `./links → /srv/links:ro` (gateway)	bind mount, read-only	New link files dropped into `links/` on the host are immediately usable by name (`link=my.txt`) and by URL, without rebuilding. Read-only because neither container should change them.

MOUNT	TYPE	REASON
`gateway-logs → /var/log/nginx`	named volume	Keeps the gateway's access log across `docker compose down` / `up` (verified: Figure 13). The API's own log goes to stdout and is read with `docker compose logs`.
`data/locations.csv`	copied into the image	The dataset is fixed and is what the API is evaluated on, so it belongs to the image version. It can still be replaced at run time by mounting a file and setting `LOCATIONS_CSV`.

2.5 Environment variables

VARIABLE	WHERE	DEFAULT	EFFECT
`PORT`	api	`8000`	gunicorn bind port (read by `gunicorn.conf.py` and the health check)
`WORKERS`, `THREADS`	api	`4`, `2`	gunicorn processes and threads per process; set from `API_WORKERS` / `API_THREADS` in `.env`
`LOCATIONS_CSV`	api	`/app/data/locations.csv`	Dataset path (already supported by `app.py`)
`LINKS_DIR`	api	`/app/links`	Folder searched when `link` is a bare filename
`PYTHONUNBUFFERED`, `PYTHONDONTWRITEBYTECODE`	api	`1`	Logs appear in `docker logs` immediately; no `.pyc` writes into the read-only app folder
`PIP_DEFAULT_TIMEOUT`, `PIP_RETRIES`	builder	`120`, `10`	Build-time only: survive the slow connection to PyPI (Error 2)
`API_HOST`, `API_PORT`	gateway	`api`, `8000`	Substituted into the nginx `upstream` block at start-up
`MAX_UPLOAD`	gateway	`64m`	`client_max_body_size`; nginx's default of 1 MB would reject large link uploads
`HOST_PORT`	compose (.env)	`8080`	Host side of the gateway's port mapping

2.6 Start-up order, health and restarts

Each image defines a `HEALTHCHECK`: the API runs `GET /health` every 15 s and the gateway fetches `/gateway-health`. The gateway's `depends_on` uses `condition: service_healthy`, so nginx starts only after the API has loaded the dataset and is answering. The output of `docker compose up` shows *api Waiting → Healthy* and only then *gateway Starting* (Figure 4). Both services use `restart: unless-stopped`. `CMD` is in exec (JSON) form so that gunicorn is PID 1 and receives `SIGTERM` directly: `docker compose stop api` finishes in 0.9 s instead of waiting for the 10 s kill timeout (Error 3).

3 Steps followed, with screenshots

All steps were run on my own laptop (Windows 11 Home, 20 logical CPUs, 8 GB given to Docker's WSL 2 VM) from Git Bash in the project folder; the same commands work unchanged in PowerShell. Every command's output was captured to a log file as it ran (all logs are in `report/docker/logs/`), and Figures 2–13 show that output, unedited, in a terminal frame; `⋮` marks where lines of a long build log were cut for space. Section 3.10 has direct captures of real Git Bash windows running the main steps.

3.1 Install Docker and check the installation

Docker on Windows runs Linux containers inside a lightweight VM managed by WSL 2, so the installation has two parts. These are the steps followed on this laptop:

1. **Check virtualisation.** Hardware virtualisation (Intel VT-x on this laptop's Core i5-13500HX) must be enabled in the BIOS. Task Manager → Performance → CPU shows *Virtualization: Enabled*.
2. **Install WSL 2.** In an administrator PowerShell: `wsl --install`. This enables the *Virtual Machine Platform* and *Windows Subsystem for Linux* features, installs the WSL kernel and an Ubuntu distribution, and needs one reboot. `wsl --version` then confirms WSL 2.6 with Linux kernel 6.6.
3. **Install Docker Desktop.** Download *Docker Desktop Installer.exe* from `docs.docker.com/desktop/setup/install/windows-install/`, run it, and keep *Use WSL 2 instead of Hyper-V* ticked. It installs to `C:\Program Files\Docker\Docker` and adds the `docker` CLI, BuildKit (`buildx`) and the Compose v2 plugin to the PATH.
4. **First start.** Launch Docker Desktop and accept the subscription agreement (free for personal and education use). On first start it creates its own WSL distribution, `docker-desktop`, which runs the Docker Engine, and a CLI context `desktop-linux` that connects to it through a named pipe. If the CLI is used before the engine is running, it fails with the named-pipe error in Error 1.
5. **Verify.** `docker version` must show both a client and a server, and `docker run hello-world` must print its greeting.

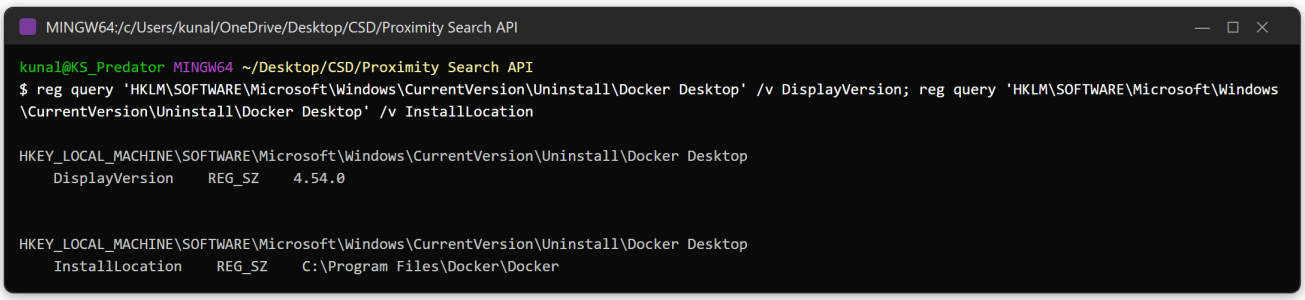


Figure 2a. Docker Desktop 4.54.0 is registered as an installed program, in C:\Program Files\Docker\Docker.

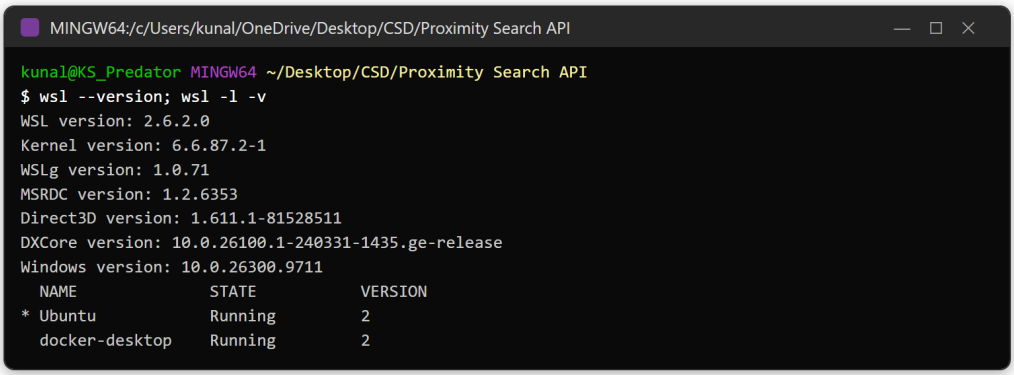


Figure 2b. WSL 2 is installed, and the docker-desktop distribution (the VM that runs the engine) is running next to Ubuntu.

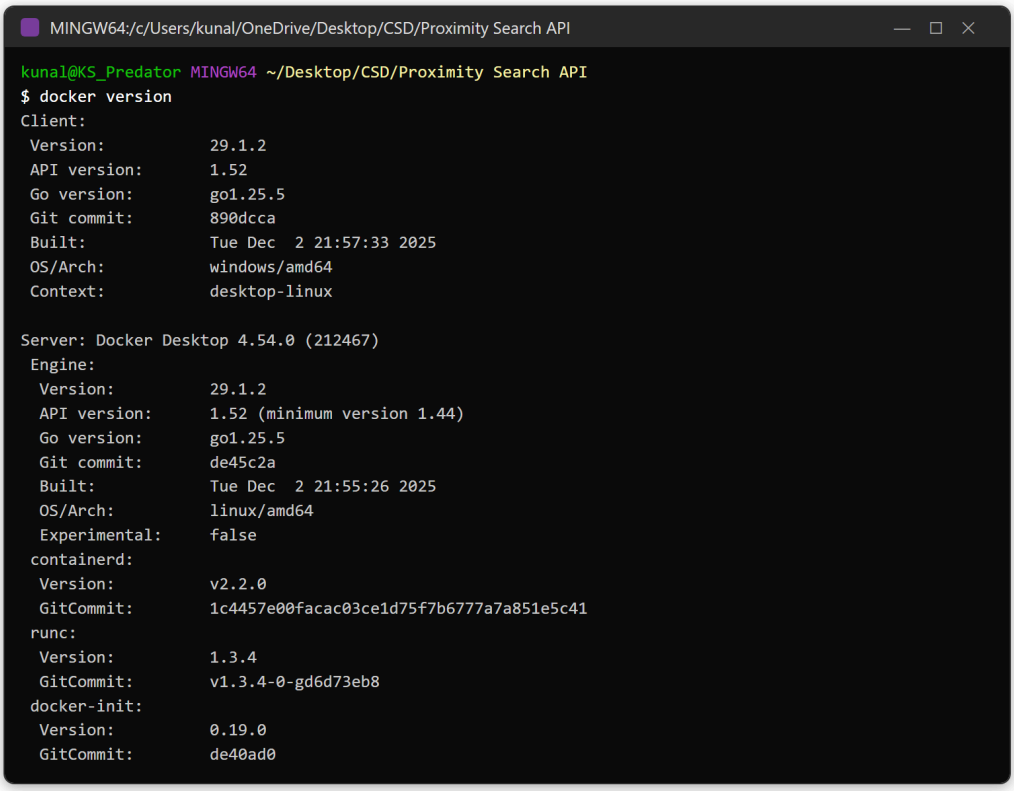


Figure 2c. `docker version` : a Windows client talking to a Linux server (Docker Desktop 4.54.0, Engine 29.1.2, containerd 2.2, runc 1.3).

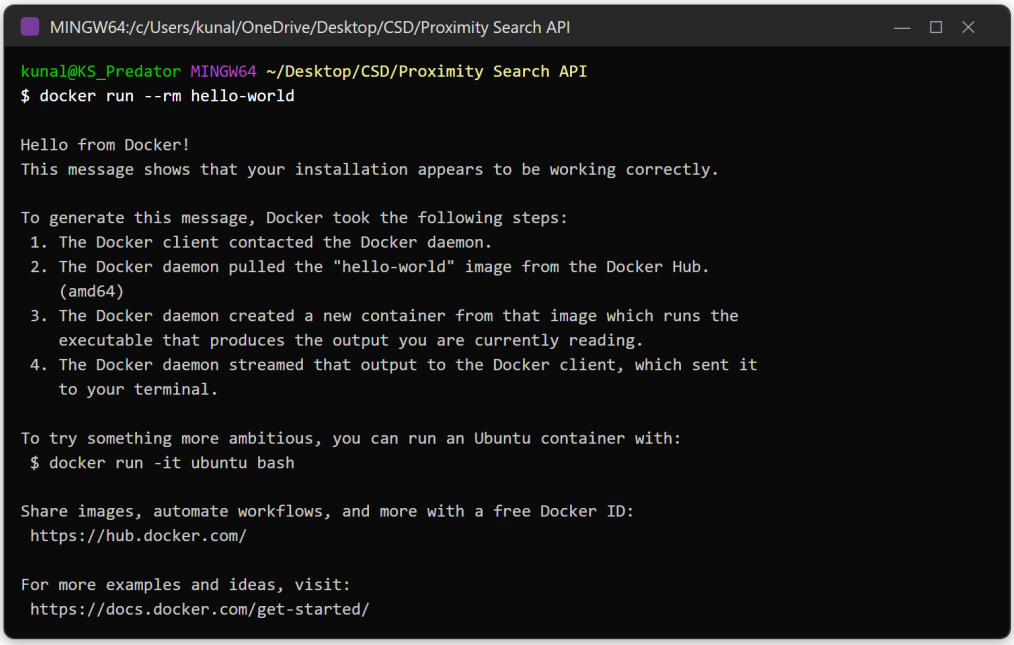


Figure 2d. The official installation test. The first attempt failed while downloading the image (see Error 6); after `docker pull hello-world` it ran correctly.

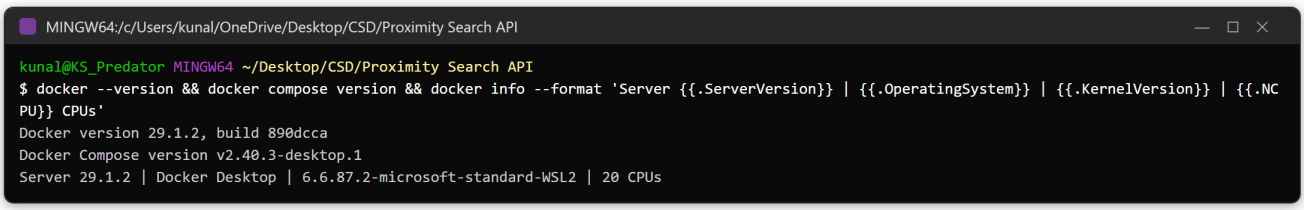


Figure 2e. Client, Compose and engine versions in one line; the engine runs on the WSL 2 kernel.

3.2 Write the Dockerfiles and the compose file, then validate

The files listed in Section 1 were written (full text in the Appendix). `docker compose config` parses the YAML, fills in the `.env` values and reports errors without building anything:

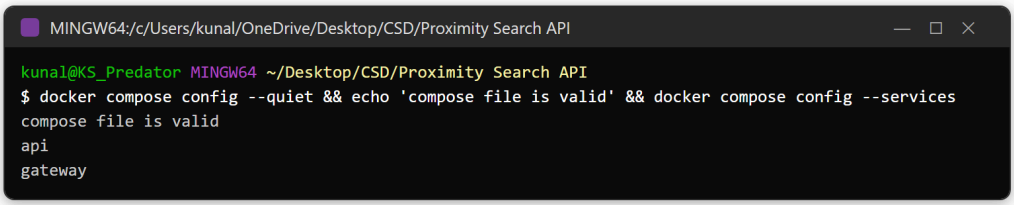


Figure 3. The compose file is valid and defines two services.

3.3 Build the test image and run the tests in a container

Before starting the stack, the `test` stage was built and the existing 23 pytest tests (which compare the engine against Floyd–Warshall and exhaustive-search references) were run inside it. This checks that the code behaves the same in the Linux container as it did on Windows. The first attempts hit Errors 2, 4, 5, 10, 11 and 12; the final run:

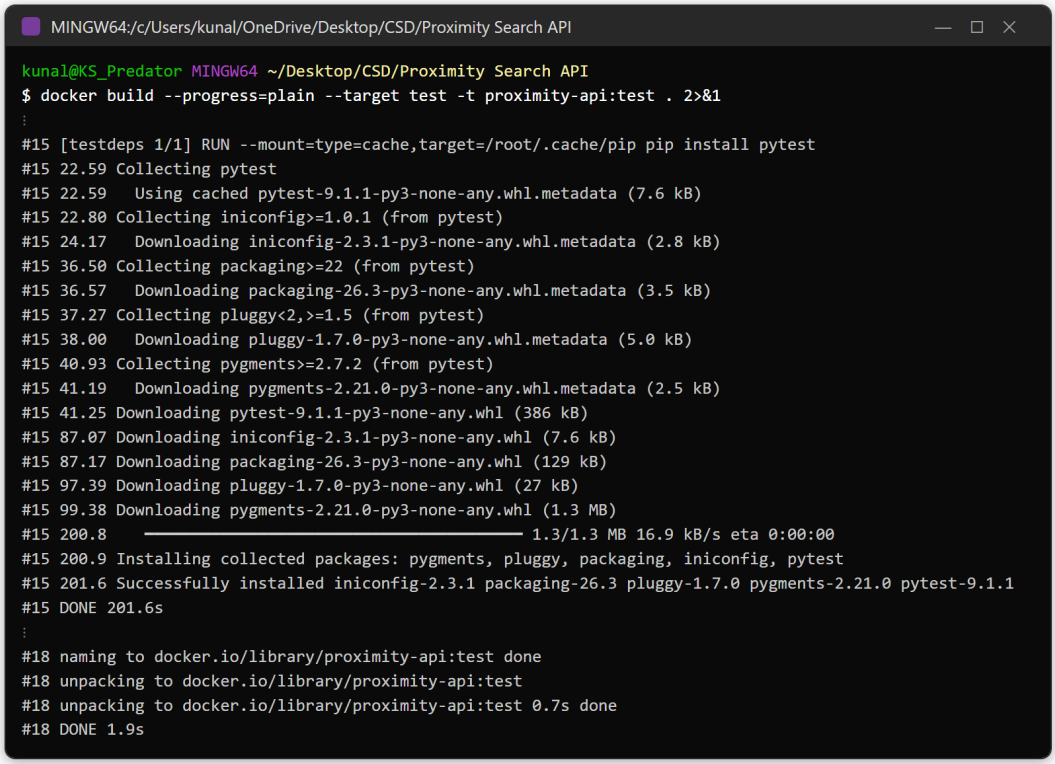


Figure 4a. Building the test target with the final Dockerfile. pytest's metadata comes from the BuildKit cache mount filled by the failed attempt ("Using cached"), and the remaining downloads finish at about 17 kB/s (Error 12).

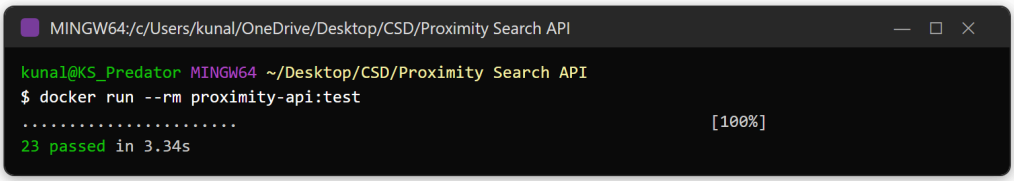


Figure 4b. All 23 tests pass inside the container, with no warnings.

3.4 Build and start the whole stack with one command

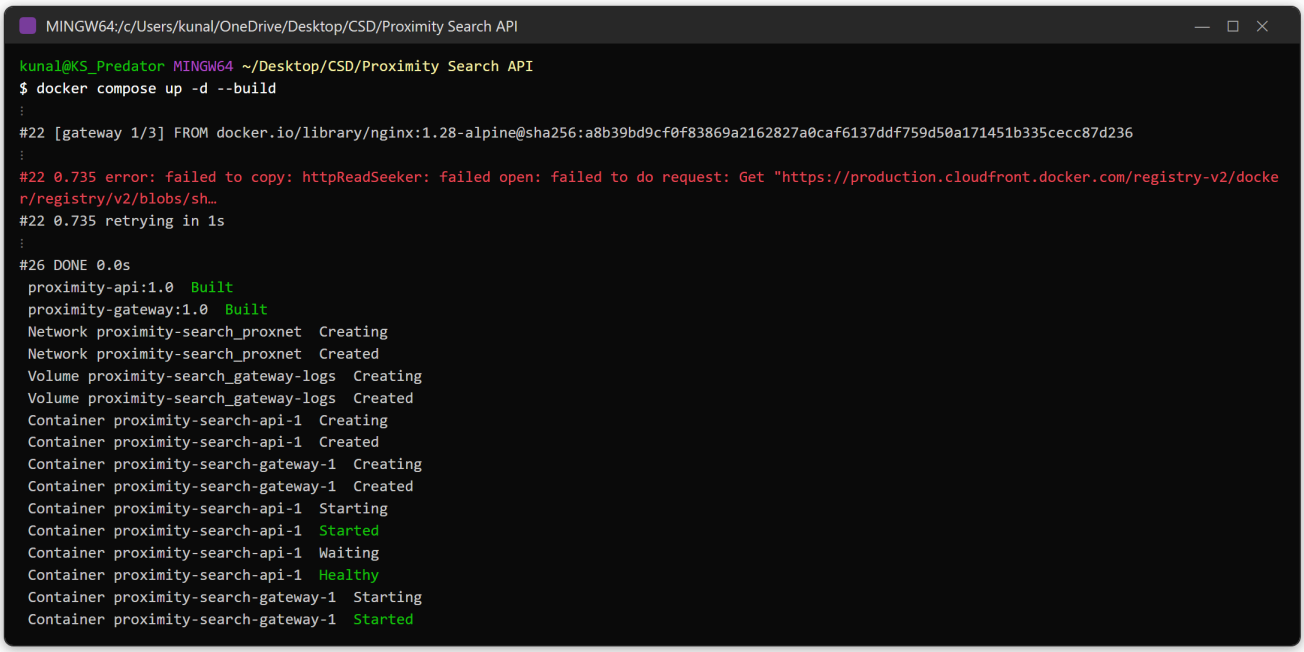


Figure 5. `docker compose up -d --build` builds both images, creates the network and the volume, waits for the API to become healthy, then starts the gateway. The red lines are a CDN download error that BuildKit retried on its own (Error 6).

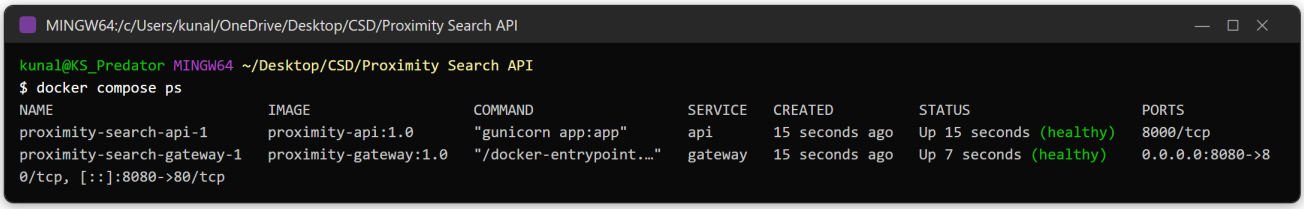


Figure 6. Both containers are up and healthy. Only the gateway has a host port (8080 → 80); the API shows only 8000/tcp.

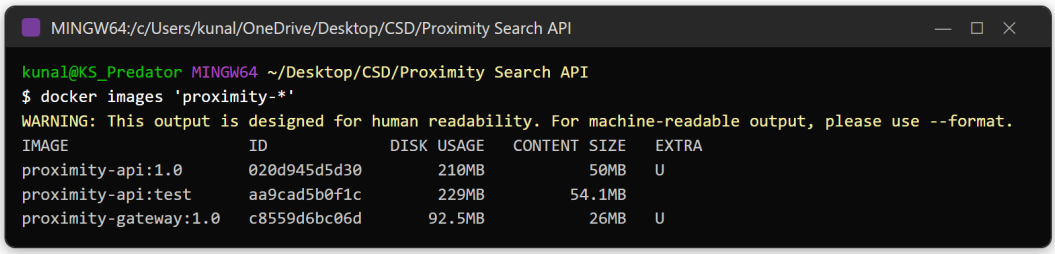


Figure 7. The two images built by Compose, plus the test image.

3.5 Use the API through the gateway



Figure 8a. Health endpoints: the API's /health (proxied by nginx) and nginx's own /gateway-health.

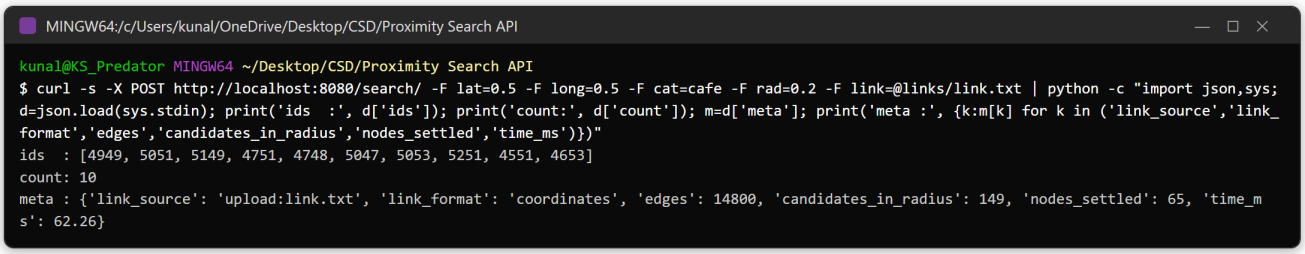


Figure 8b. The README's example query, uploading link.txt as a file. The IDs match the Lab 7 deployment exactly.

```
http://localhost:8080/search/?
lat=0.5&long=0.5&cat=cafe&rad=0.2&link=http://gateway/files/link.txt
```

Pretty-print


```
{"ids":[4949,5051,5149,4751,4748,5047,5053,5251,4551,4653],"count":10,"results":
[{"rank":1,"id":4949,"lat":0.494949,"long":0.484848,"category":"cafe","grid_distance":0.010101,"euclidean_distance":0.015972,
"hops":1},
{"rank":2,"id":5051,"lat":0.505051,"long":0.505051,"category":"cafe","grid_distance":0.020204,"euclidean_distance":0.007143,"
hops":2},
{"rank":3,"id":5149,"lat":0.515152,"long":0.484848,"category":"cafe","grid_distance":0.030304,"euclidean_distance":0.021428,"
hops":3},
{"rank":4,"id":4751,"lat":0.474747,"long":0.505051,"category":"cafe","grid_distance":0.030304,"euclidean_distance":0.025753,"
hops":3},
{"rank":5,"id":4748,"lat":0.474747,"long":0.474747,"category":"cafe","grid_distance":0.040404,"euclidean_distance":0.035713,"
hops":4},
{"rank":6,"id":5047,"lat":0.505051,"long":0.464646,"category":"cafe","grid_distance":0.040405,"euclidean_distance":0.035713,"
hops":4},
{"rank":7,"id":5053,"lat":0.505051,"long":0.525253,"category":"cafe","grid_distance":0.040406,"euclidean_distance":0.025753,"
hops":4},
{"rank":8,"id":5251,"lat":0.525253,"long":0.505051,"category":"cafe","grid_distance":0.040406,"euclidean_distance":0.025753,"
hops":4},
{"rank":9,"id":4551,"lat":0.454545,"long":0.505051,"category":"cafe","grid_distance":0.050506,"euclidean_distance":0.045735,"
hops":5},
{"rank":10,"id":4653,"lat":0.464646,"long":0.525253,"category":"cafe","grid_distance":0.060607,"euclidean_distance":0.043447,
"hops":6}],"query":{"lat":0.5,"long":0.5,"cat":"cafe","rad":0.2,"k":10},"meta":{"start_node":
{"id":4950,"lat":0.494949,"long":0.494949},"link_source":"url","graph":"link-
file","edges":14800,"link_format":"coordinates","link_lines_ignored":0,"link_endpoints_snapped":0,"link_endpoints_unknown":0,
"candidates_in_radius":149,"nodes_settled":65,"time_ms":94.58}}
```


Figure 8c. The same query in Chrome, this time with `link` given as a URL that points at the gateway. The API container fetched the link file from the gateway container by its service name (`"link_source":"url"`).

3.6 Verify the network, isolation and configuration

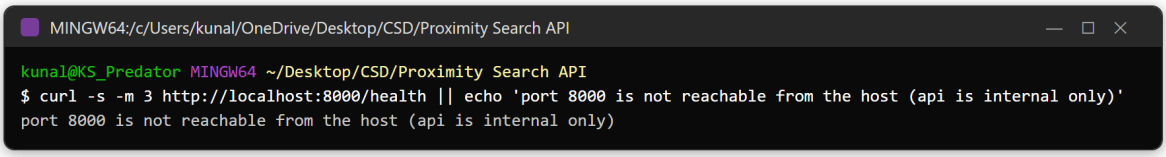


Figure 9. The API's port is not reachable from the host; all traffic has to go through nginx.

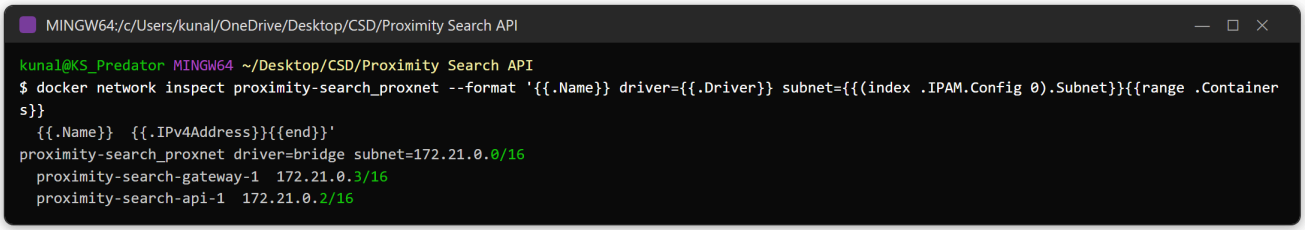


Figure 10a. The proxnet bridge network and the address of each container.

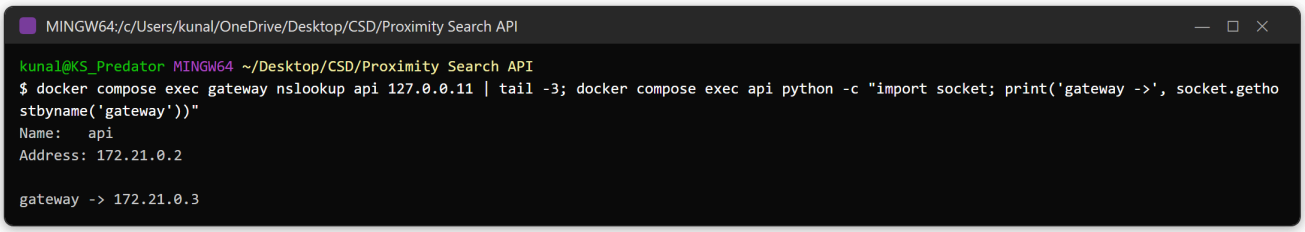


Figure 10b. Docker's embedded DNS resolves service names in both directions.

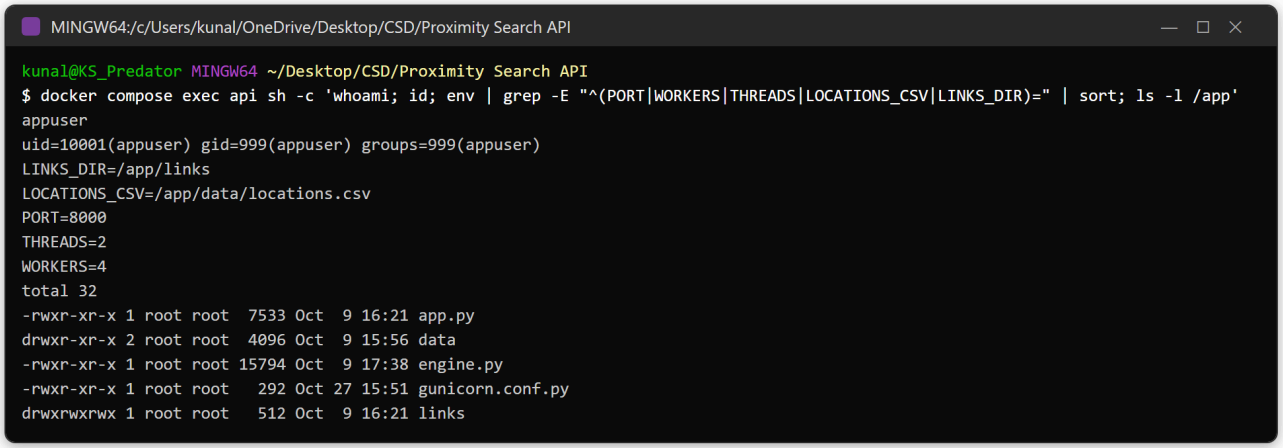


Figure 10c. Inside the API container: the process runs as the non-root appuser, the environment variables are set, and the app files belong to root.

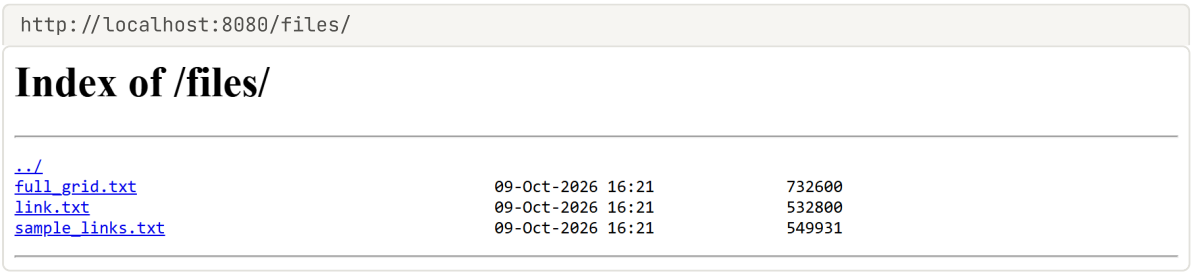


Figure 10d. The read-only links/ bind mount, served by nginx at /files/.

3.7 Check that the container gives the same answers as the native code

A short script sent 200 random queries (random position, category and radius, seed 42) through the gateway and compared each top-10 list with the one from `engine.py` run directly on Windows:

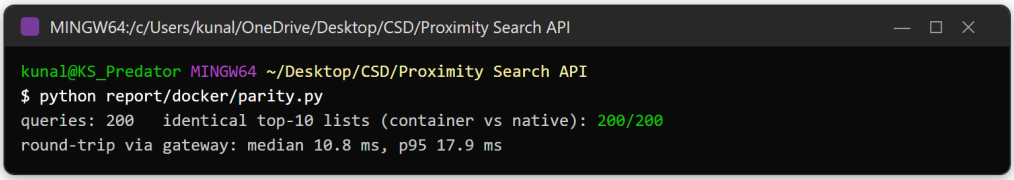


Figure 11. 200 of 200 identical results; median round trip 10.8 ms through nginx and gunicorn.

3.8 Logs, resource usage and the log volume

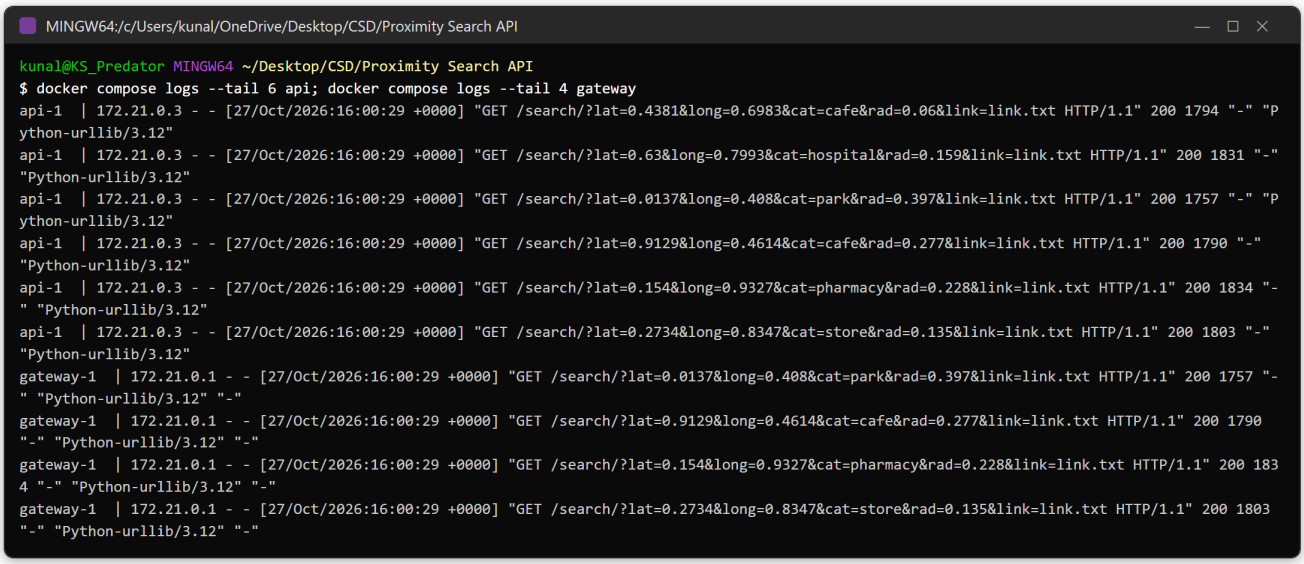


Figure 12a. `docker compose logs`: requests reach gunicorn from the gateway's address (172.21.0.3) and nginx from the host side of the bridge (172.21.0.1).

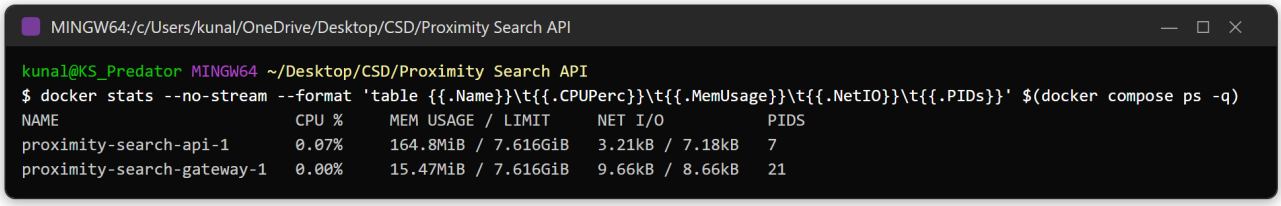


Figure 12b. Live resource use of the two containers: the API takes about 165 MiB (four workers, each holding the dataset), nginx about 15 MiB.

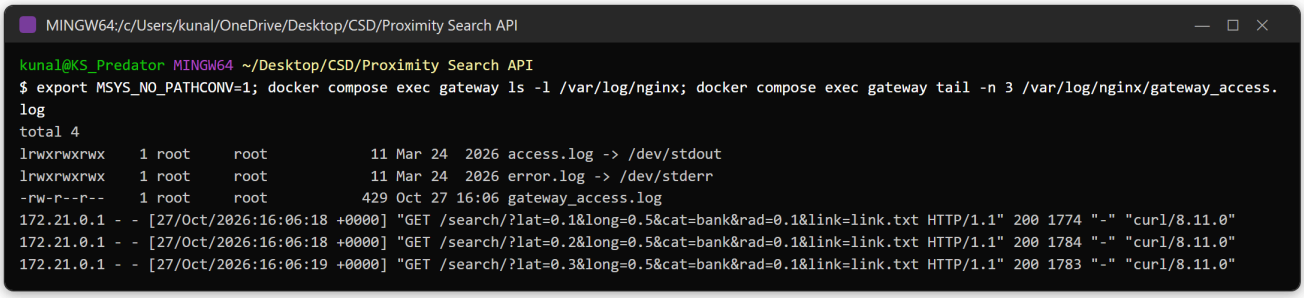


Figure 13a. After the fix for Error 7, the gateway writes a real log file into the named volume.

3.9 Stop, tear down and start again

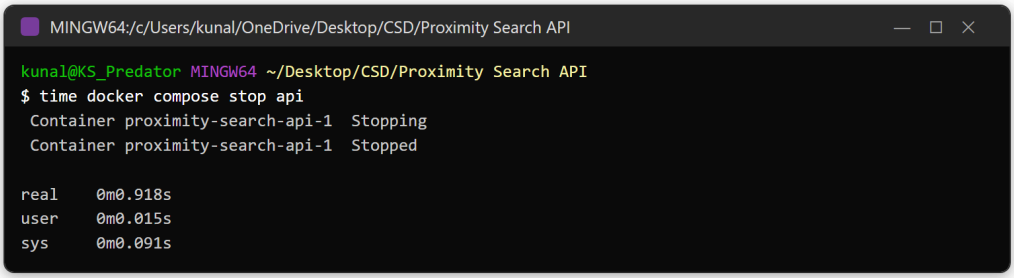


Figure 13b. A graceful stop takes under a second, because gunicorn is PID 1 and handles SIGTERM.

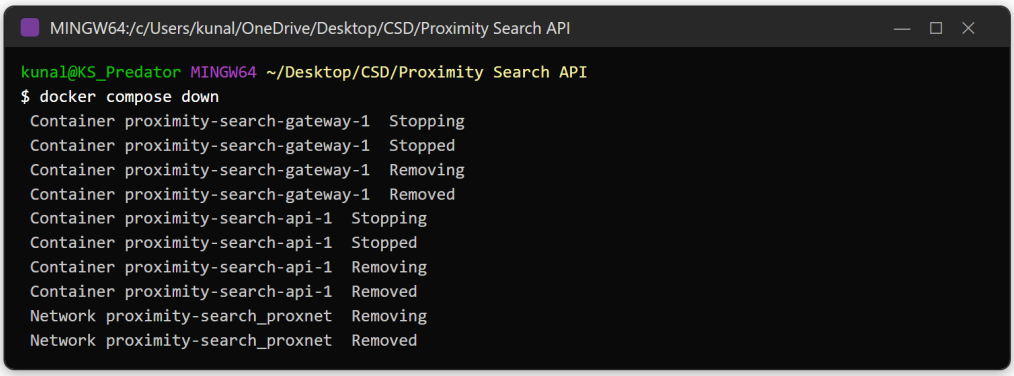


Figure 13c. `docker compose down` removes the containers and the network but keeps the named volume.

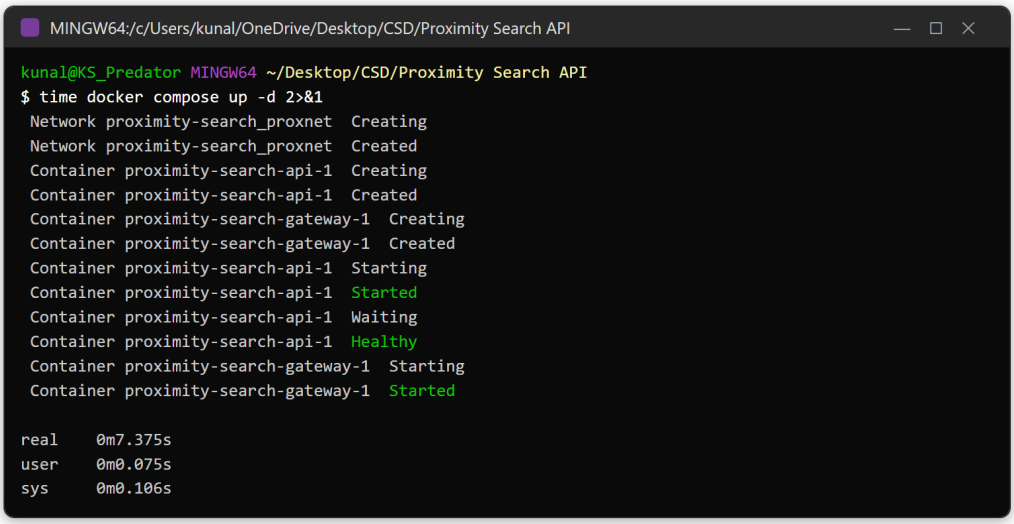


Figure 13d. Starting again from the built images takes 7.4 s, including the API's health check.

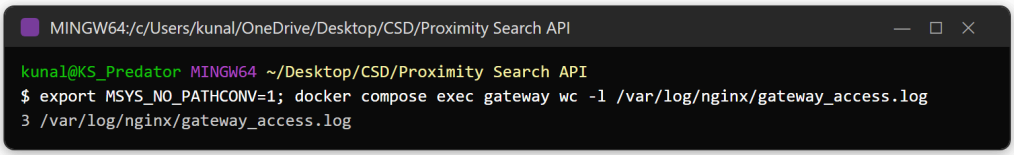


Figure 13e. The access log survived `down` / `up`: the volume persists.

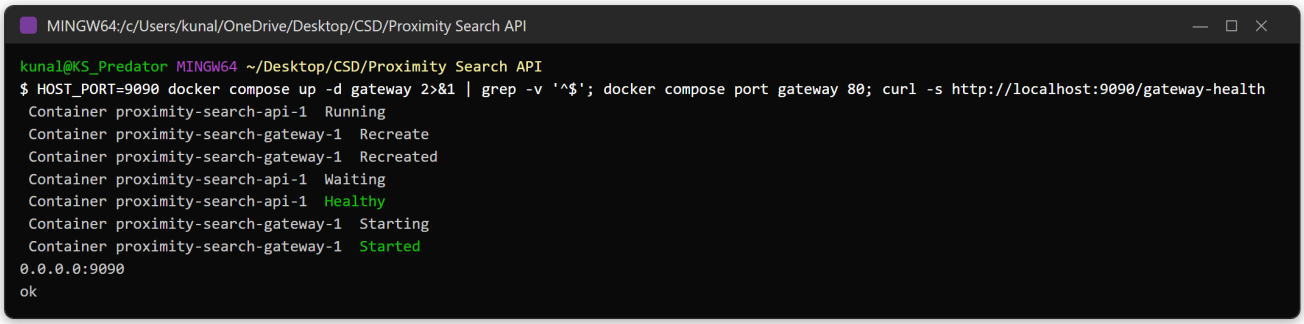


Figure 13f. Overriding an environment variable from the shell: HOST_PORT=9090 republishes the gateway on port 9090 without editing any file.

3.10 Real terminal windows

To show the steps as they look on screen, the main build, run and verify commands were run once more in Git Bash windows, and each window was captured directly. The `$` lines come from bash's own command trace (`set -x`).

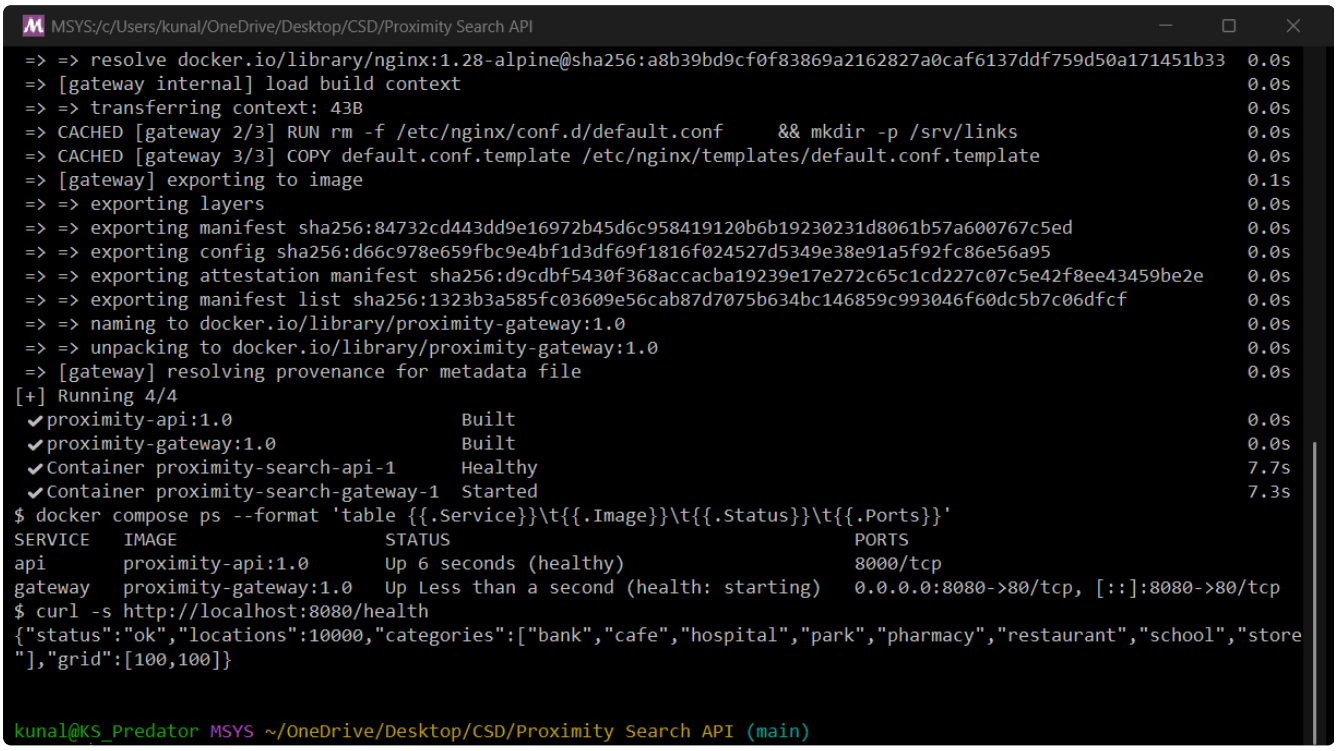


Figure 14a. Git Bash window: `docker compose up -d --build` finishing (images built, api Healthy, then gateway Started), the status table, and the health endpoint through the gateway.

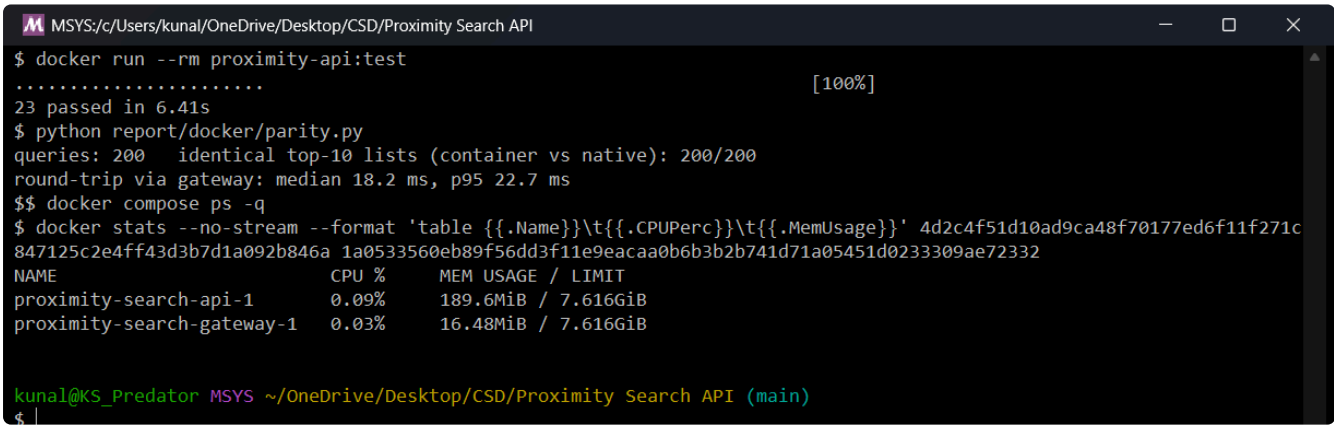


Figure 14b. Git Bash window: the test suite inside the test image (23 passed), the 200-query check against the native engine (200/200), and the containers' live resource use.

4 Errors faced and how they were resolved

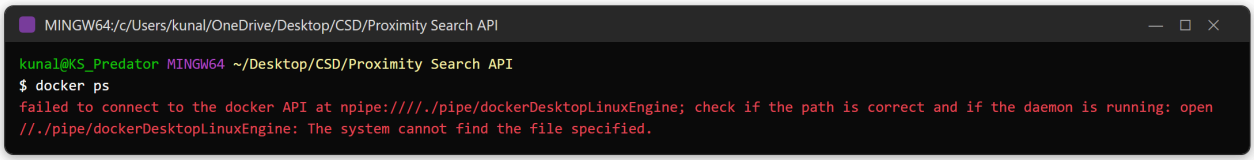
These are the problems that actually came up, in the order they appeared, each with the evidence captured at the time.

1 Docker engine not running

SYMPTOM — `docker ps` → `failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine … The system cannot find the file specified.`

CAUSE — On Windows the `docker` CLI is only a client. It talks to the engine inside Docker Desktop's WSL 2 VM through a named pipe, and that pipe exists only while Docker Desktop is running.

FIX — Started Docker Desktop and polled `docker info` until the engine answered (Figures 2c and 2e).

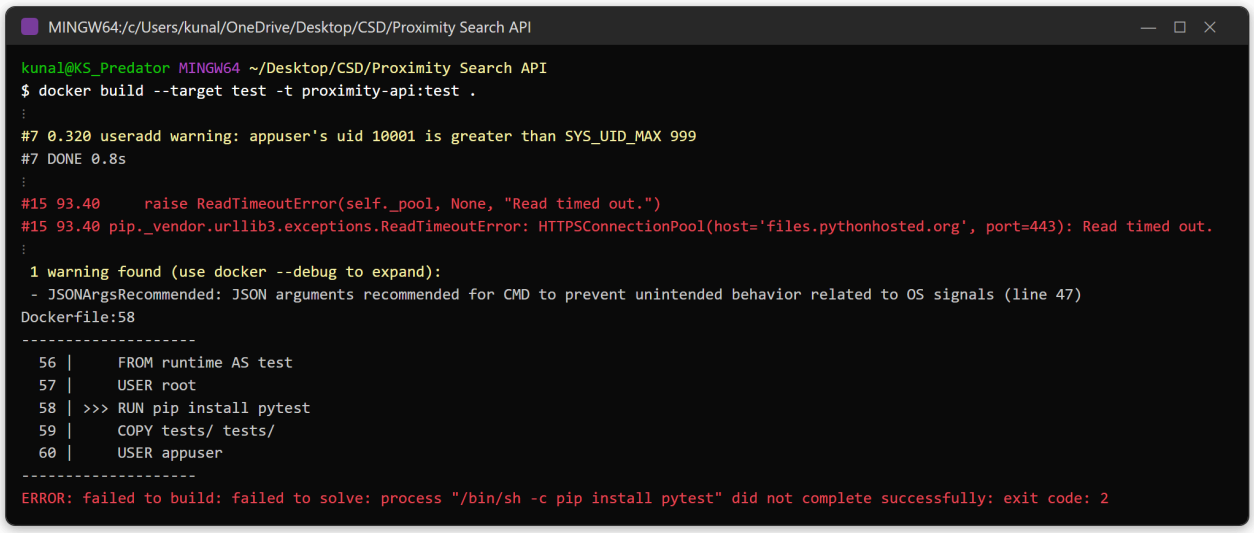


2 `pip install` timed out during the build (ReadTimeoutError)

SYMPTOM — The build failed after 93 s: `ReadTimeoutError: HTTPSConnectionPool(host='files.pythonhosted.org', port=443): Read timed out.` Even the successful step that installed Flask took 110 s.

CAUSE — The internet connection to PyPI's file CDN was very slow, and pip's default read timeout of 15 s was shorter than some of its stalls.

FIX — Set `PIP_DEFAULT_TIMEOUT=120` and `PIP_RETRIES=10` in the builder stage. Later builds log `WARNING: Retrying …` and then succeed instead of failing (Figure 4a).



3 Build-check warning: `JSONArgsRecommended`

SYMPTOM — BuildKit warned: `JSON arguments recommended for CMD to prevent unintended behavior related to OS signals` (visible at the bottom of the previous screenshot).

CAUSE — The first Dockerfile used a shell-form `CMD exec gunicorn --bind "0.0.0.0:${PORT}" …` so that environment variables would be expanded. Shell form runs under `/bin/sh -c`, which may not pass `SIGTERM` on to gunicorn, so `docker stop` would wait 10 s and then kill it.

FIX — Moved the settings into `gunicorn.conf.py`, which reads `PORT` / `WORKERS` / `THREADS` from the environment (gunicorn loads it automatically from the working directory), and changed to `CMD ["gunicorn", "app:app"]`. Stopping now takes 0.9 s (Figure 13b).

4 `useradd` warning: uid above SYS_UID_MAX

SYMPTOM — `useradd warning: appuser's uid 10001 is greater than SYS_UID_MAX 999` (top of the screenshot for Error 2).

CAUSE — `--system` asks for an account in the system range (< 1000), but I had also given a fixed high uid. A high uid is deliberate, since it cannot collide with any account on the host.

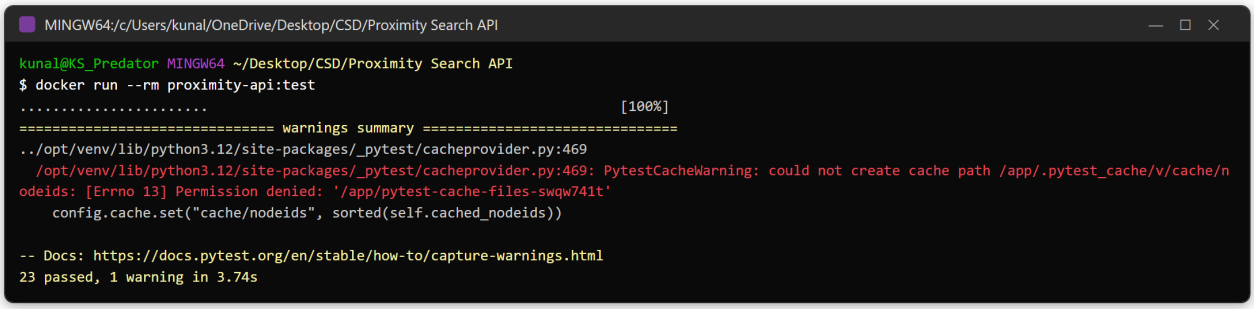
FIX — Dropped `--system` and gave the account a `/usr/sbin/nologin` shell instead: `useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser`.

5 pytest could not write its cache inside the container

SYMPTOM — The tests passed, but with `PytestCacheWarning: could not create cache path /app/.pytest_cache … [Errno 13] Permission denied`.

CAUSE — The container runs as `appuser` and `/app` is owned by root. This is intended: the running app must not be able to modify its own code.

FIX — Kept the permissions and turned off pytest's cache plugin in the test image: `CMD ["python", "-m", "pytest", "-q", "-p", "no:cacheprovider"]`.

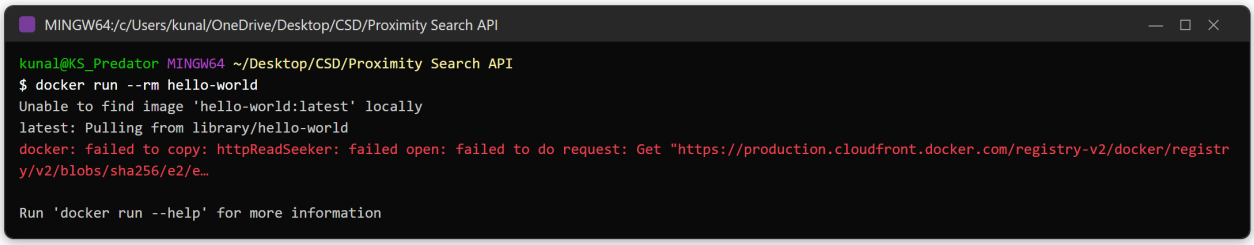


6 Docker Hub layer download: `EOF`

SYMPTOM — While pulling `nginx:1.28-alpine`: `error: failed to copy: httpReadSeeker: failed open … production.cloudfront.docker.com … EOF`.

CAUSE — The connection to Docker Hub's CDN dropped partway through a layer download, again caused by the unreliable network.

FIX — During the build none was needed: BuildKit logged `retrying in 1s`, and the pull and the build then succeeded (Figure 5). The same `EOF` later hit `docker run --rm hello-world` (below), and that command does *not* retry: it exited with code 125. I re-ran the download on its own with `docker pull hello-world`, which succeeded, and `docker run hello-world` then worked (Figure 2d). So BuildKit retries registry downloads, but plain `docker run` and `RUN pip install` inside a build do not (Error 2).

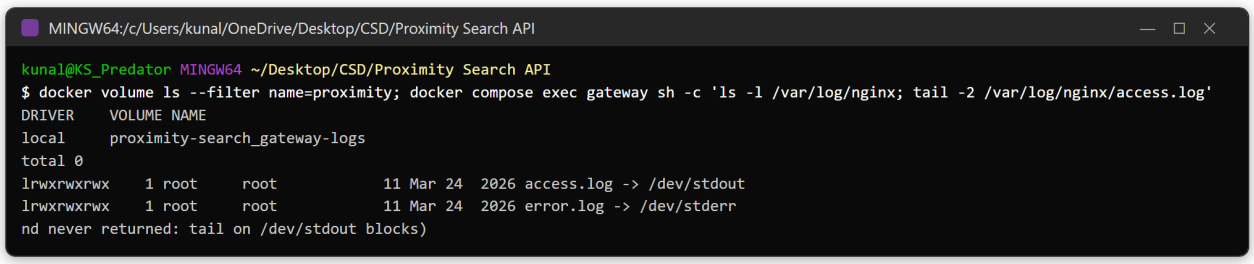


7 The log volume held nothing useful, and reading it hung the terminal

SYMPTOM — `/var/log/nginx` in the named volume held only two symlinks, and `tail /var/log/nginx/access.log` never returned.

CAUSE — The official nginx image links `access.log → /dev/stdout` and `error.log → /dev/stderr` so that logs go to `docker logs`. When the empty named volume was first mounted, Docker copied those symlinks into it, so no log was ever written to disk, and `tail` on `/dev/stdout` blocks forever.

FIX — Added a second `access_log /var/log/nginx/gateway_access.log;` next to `access_log /dev/stdout;`. Logs now go to both `docker logs` and the volume (Figures 13a and 13e).

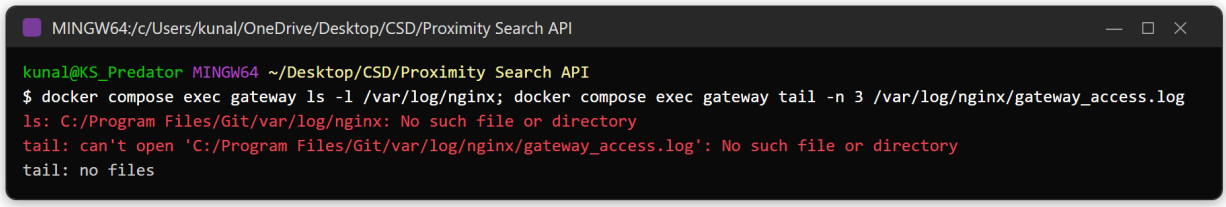


8 Git Bash rewrote container paths (MSYS path conversion)

SYMPTOM `docker compose exec gateway ls -l /var/log/nginx` → `ls: C:/Program Files/Git/var/log/nginx: No such file or directory`.

CAUSE Git Bash (MSYS2) converts any argument that looks like a Unix path into a Windows path before running a Windows program such as `docker.exe`. It treated `/var/log/nginx` as relative to Git's install folder, even though the path was meant for inside the Linux container.

FIX Set `export MSYS_NO_PATHCONV=1` for commands that pass container paths (Figure 13a). The same commands work unchanged in PowerShell, which does no such conversion.

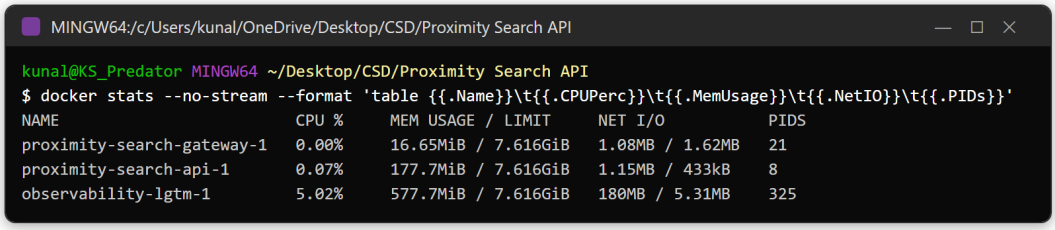


9 `docker stats` listed a container from another project

SYMPTOM The stats table also showed `observability-lgtm-1`, a container from unrelated work on the same laptop.

CAUSE `docker stats` covers every running container on the engine, not just the current Compose project. This happens on a personal machine that is also used for other projects.

FIX Limited it to this project with `docker stats --no-stream $(docker compose ps -q)` (Figure 12b).

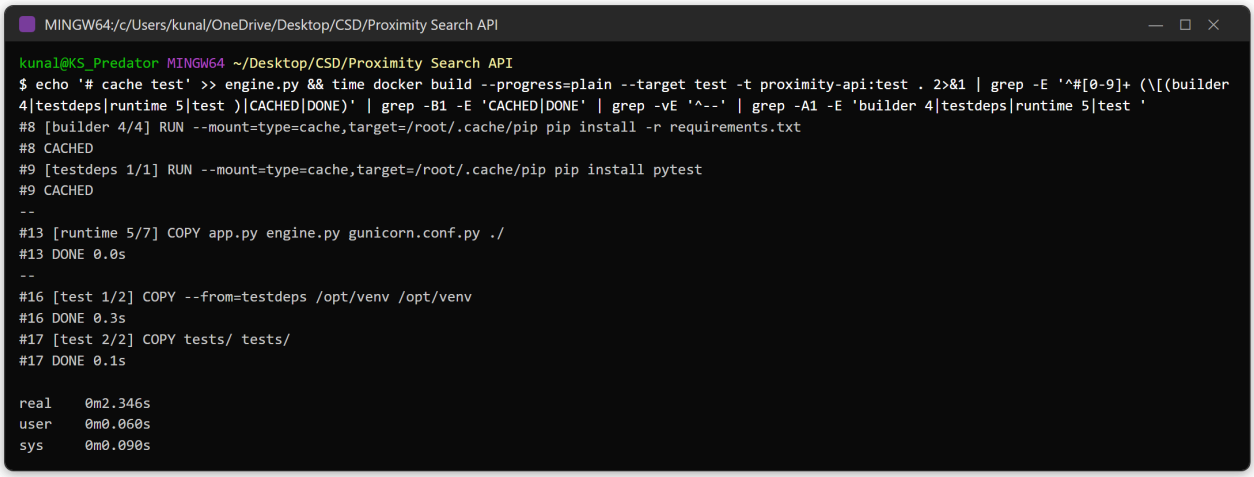


10 Every code change made the test build download pytest again (and time out)

SYMPTOM — After a one-line change to the runtime stage (the `useradd` fix), rebuilding the test image ran `pip install pytest` again and failed with the same `ReadTimeoutError` after 230 s.

CAUSE — Layer-cache ordering. The `test` stage was `FROM runtime` followed by `RUN pip install pytest`, so any change in `runtime` (including any change to the source code) invalidated the pytest layer and forced a fresh download.

FIX — Added a `testdeps` stage (`FROM builder`) that installs pytest into the venv. That stage depends only on `requirements.txt`; `test` now copies the finished venv from it. Both pip steps also use a BuildKit cache mount, `RUN --mount=type=cache,target=/root/.cache/pip`, so downloaded wheels are kept between builds without ending up in an image. To check it, I appended a comment to `engine.py` and rebuilt: the source layer was rebuilt, the pip layers stayed `CACHED`, and the whole test image rebuilt in 2.3 s (the edit was then reverted).

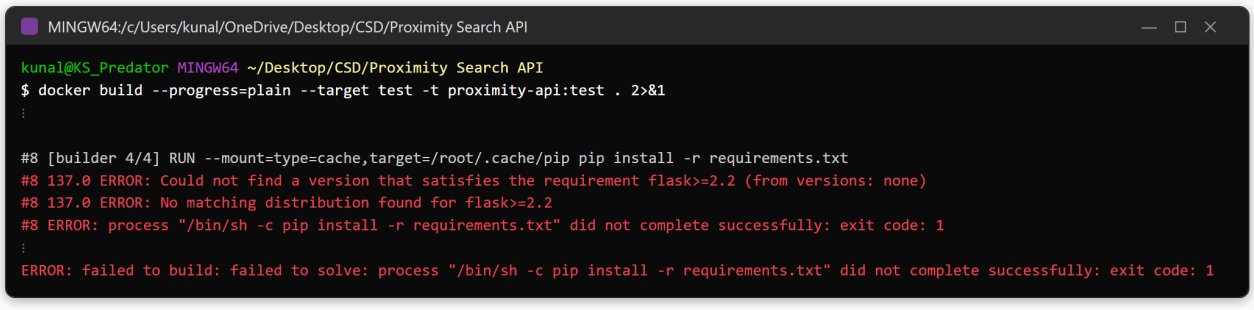


11 `No matching distribution found for flask⩾2.2`

SYMPTOM — On the next build, pip could not even read the package index: `Could not find a version that satisfies the requirement flask⩾2.2 (from versions: none)`, after 137 s.

CAUSE — "from versions: none" means pip got no index page at all. The message looks like a version problem, but the cause was the network. To confirm it, I fetched `https://pypi.org/simple/flask/` from the host (HTTP 200 in 0.4 s) and from a throwaway `docker run --rm python:3.12-slim` container (39 kB in 1.5 s). Both worked, so DNS and the index were reachable from containers, and the failure was intermittent.

FIX — Re-ran the build unchanged. It got past the index this time (Flask installed after one retry), but the next download stalled again. That led to the real cause, Error 12.



12 Downloads inside Docker were about 100× slower than on the host

SYMPTOM

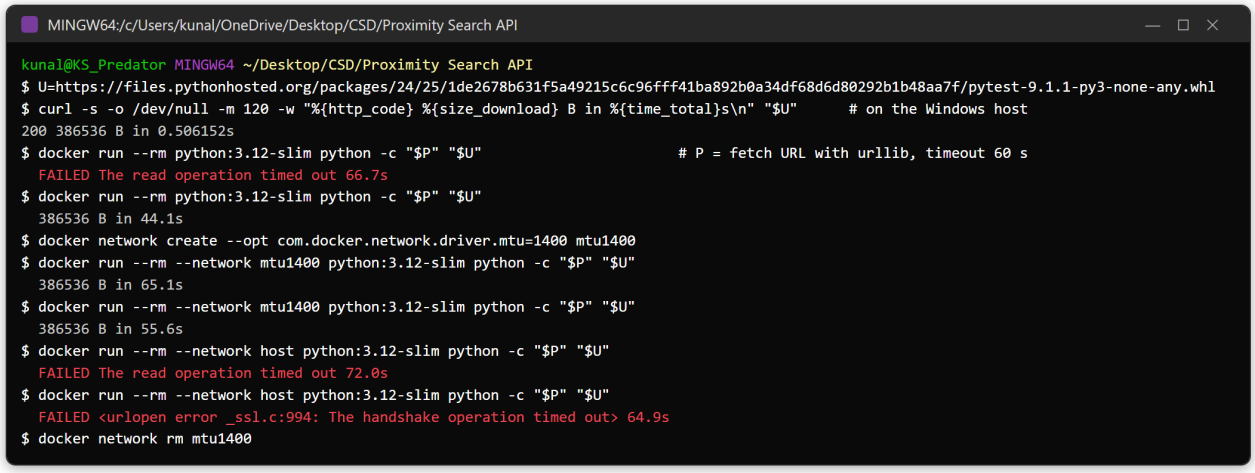
Even with retries, `pip install pytest` failed once more after 509 s with `ReadTimeoutError`. Small requests (index pages) worked, but files of a few hundred kB stalled.

DIAGNOSIS

I downloaded the same 386 kB pytest wheel in different places. From the Windows host it took **0.5 s**. From a container on the default bridge it either timed out or took **44 s**. I suspected a path-MTU problem (common with WSL 2 and VPNs), so I created a bridge network with MTU 1400: still 55–65 s, so MTU was not the cause. With `--network host` (the VM's own network stack) it also failed. The bottleneck is therefore Docker Desktop's VM network path on this laptop, not the internet, the image or the Dockerfile. The successful build logged 16.9 kB/s.

FIX

The build was made tolerant of the slow link instead of depending on it: the pip timeout and retries from Error 2, and the cache mount from Error 10. The mount meant that the failed attempt's downloads were reused (`Using cached pytest-9.1.1…`), and the next build succeeded (Figure 4a). After that, every rebuild hits the layer cache and needs no network at all. The proper fix would be in Docker Desktop's network settings (or a restart of its VM). I left that alone because other containers on this machine depend on the same engine.



5 Docker commands used

5.1 In the order they were run

Every command that called Docker, taken from the captured terminal logs. Pipes into `grep` / `tail` that only trimmed the output for the screenshots have been removed.

```
docker ps
docker --version && docker compose version && docker info --format 'Server {{.ServerVersion}} |
{{.OperatingSystem}} | {{.KernelVersion}} | {{.NCPU}} CPUs'
docker compose config --quiet && echo 'compose file is valid' && docker compose config --services
docker build --target test -t proximity-api:test .
docker build --progress=plain --target test -t proximity-api:test .
docker run --rm proximity-api:test
docker compose up -d --build
docker compose ps
docker network inspect proximity-search_proxnet --format '{{.Name}} driver={{.Driver}} subnet={{(index
.IPAM.Config 0).Subnet}}{{range .Containers}}
  {{.Name}}  {{.IPv4Address}}{{end}}'
docker compose exec gateway nslookup api 127.0.0.11; docker compose exec api python -c "import socket;
print('gateway →', socket.gethostbyname('gateway'))"
docker compose exec api sh -c 'whoami; id; env | grep -E
"^(PORT|WORKERS|THREADS|LOCATIONS_CSV|LINKS_DIR)=" | sort; ls -l /app'
python report/docker/parity.py
docker compose logs --tail 6 api; docker compose logs --tail 4 gateway
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}\t{{.PIDs}}'
docker volume ls --filter name=proximity; docker compose exec gateway sh -c 'ls -l /var/log/nginx; tail
-2 /var/log/nginx/access.log'
docker compose up -d --build gateway
docker compose exec gateway ls -l /var/log/nginx; docker compose exec gateway tail -n 3
/var/log/nginx/gateway_access.log
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.NetIO}}\t{{.PIDs}}'
$(docker compose ps -q)
export MSYS_NO_PATHCONV=1; docker compose exec gateway ls -l /var/log/nginx; docker compose exec gateway
tail -n 3 /var/log/nginx/gateway_access.log
time docker compose stop api
docker compose down
time docker compose up -d
export MSYS_NO_PATHCONV=1; docker compose exec gateway wc -l /var/log/nginx/gateway_access.log
docker compose build --progress=plain api
docker compose up -d && docker compose ps
docker build --progress=plain --target test -t proximity-api:test .
docker run --rm python:3.12-slim python -c "$P" "$U"
docker network create --opt com.docker.network.driver.mtu=1400 mtu1400
docker run --rm --network mtu1400 python:3.12-slim python -c "$P" "$U"
docker run --rm --network host python:3.12-slim python -c "$P" "$U"
docker network rm mtu1400
HOST_PORT=9090 docker compose up -d gateway; docker compose port gateway 80; curl -s
http://localhost:9090/gateway-health
docker compose up -d; docker compose port gateway 80
docker build --progress=plain --target test -t proximity-api:test .
docker run --rm proximity-api:test
docker images 'proximity-*'
echo '# cache test' >> engine.py && time docker build --progress=plain --target test -t proximity-
api:test .
docker version
docker run --rm hello-world
docker pull hello-world
docker run --rm hello-world
docker compose up -d --build
docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
docker run --rm proximity-api:test
python report/docker/parity.py
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}' $(docker compose ps -q)
```

5.2 Unique commands and their purpose

COMMAND	PURPOSE
`docker --version`	Show the Docker CLI version
`docker version`	Full client and server (engine, containerd, runc) versions: confirms the install end to end
`docker run --rm hello-world`	Official post-install test: pull a tiny image and run it
`docker pull IMAGE`	Download an image from Docker Hub without running it (used to retry a failed pull)
`docker compose version`	Show the Compose plugin version
`docker info --format '{{...}}'`	Check that the engine is reachable; show engine version, OS, kernel and CPUs
`docker ps`	List running containers (first command; showed the engine was down)
`docker compose config --quiet / --services`	Validate the compose file (after .env substitution) and list its services
`docker build --target test -t proximity-api:test .`	Build only up to the named stage of a multi-stage Dockerfile and tag the image
`docker build -q / --progress=plain`	Quiet build that prints only the image ID / full, unabbreviated build log
`docker run --rm IMAGE [CMD]`	Run a one-off container and delete it on exit (test suite, network checks)
`docker run --rm --network NET IMAGE`	Run a one-off container on a chosen network (MTU / host-network test)
`docker compose up -d --build`	Build images if needed, create network and volume, start all services detached
`docker compose up -d [SERVICE]`	Start or recreate the stack, or one service, from existing images
`docker compose build SERVICE`	Rebuild one service's image without starting it
`docker compose ps [-q]`	Status and health of the project's containers / just their IDs
`docker compose port gateway 80`	Show which host address and port a container port is published on
`docker images 'proximity-*'`	List the built images and their sizes
`docker compose exec SERVICE CMD`	Run a command inside a running service container (env, user, DNS, files)
`docker network inspect NET --format`	Show a network's driver, subnet and attached containers
`docker network create --opt …mtu=1400 NET / docker network rm NET`	Create and remove a test bridge network with a lower MTU
`docker compose logs --tail N SERVICE`	Show the last N log lines of a service
`docker stats --no-stream $(docker compose ps -q)`	One-off CPU, memory, network and PID usage of this project's containers
`docker volume ls --filter name=…`	List volumes matching a name
`docker compose stop SERVICE`	Stop a service gracefully (SIGTERM, then SIGKILL after 10 s)
`docker compose down`	Stop and remove the project's containers and network (named volumes kept)

6 What I learnt

- **Layer order decides how fast a rebuild is.** Copying `requirements.txt` and installing it before copying the source means that a code change rebuilds in seconds. Error 10 showed the reverse: one stage built on top of another inherits every cache invalidation of its parent. Stages should depend on as little as possible.

- **Multi-stage builds separate build tools from what ships.** The runtime image has the venv and the app but not pip's cache, and the test tooling lives in a target that is never deployed. One Dockerfile produces both images.
- **Exec-form `CMD` matters for shutdown.** With shell form, `/bin/sh` is PID 1 and signals may not reach the app. Moving configuration into a file read from the environment kept the command static and made a stop take 0.9 s instead of 10 s.
- **`depends_on` alone only orders container start-up.** A health check plus `condition: service_healthy` is what makes the gateway wait until the API can answer requests.
- **User-defined networks give service discovery for free.** Containers reach each other by service name through Docker's DNS. Leaving a port unpublished keeps a service internal without any firewall rules.
- **An empty named volume is filled from the image the first time it is mounted.** That is why the nginx volume held only the image's symlinks (Error 7). A volume is only useful if the process actually writes regular files into it.
- **Read error messages for what actually failed.** "No matching distribution" was a network outage, not a version conflict. Testing the same request from the host and from a throwaway container isolated the cause in two commands.
- **On Windows, Docker runs Linux containers inside a WSL 2 VM.** The CLI talks to it through a named pipe (Error 1), and the shell can rewrite arguments before Docker sees them (Error 8).
- **Containers make the result reproducible.** The same test suite, and 200 random queries, gave identical answers in the container and natively. Anyone with Docker can now run the whole service with one command, without installing Python or nginx.

A Appendix: complete Docker files

Dockerfile

```
# Proximity Search API (Flask + gunicorn)
#
#   docker build -t proximity-api .                      → runtime image
#   docker build --target test -t proximity-api:test .   → runtime + pytest + tests
#   (stages: builder → runtime, builder → testdeps, runtime + testdeps venv → test)
#   docker run --rm proximity-api:test                   → run the test suite

# ---------- builder: install dependencies into a self-contained venv ----------
FROM python:3.12-slim AS builder

# Generous timeout/retries: downloads from PyPI timed out on a slow connection.
ENV PIP_DISABLE_PIP_VERSION_CHECK=1 \
    PIP_DEFAULT_TIMEOUT=120 \
    PIP_RETRIES=10

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Copied on its own so this layer stays cached until the dependencies change.
# The cache mount keeps downloaded wheels across builds without putting them in the image.
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

# ---------- testdeps: builder venv + pytest (only rebuilt when requirements change) ----------
FROM builder AS testdeps
RUN --mount=type=cache,target=/root/.cache/pip pip install pytest

# ---------- runtime: slim image with the venv and the app only ----------
FROM python:3.12-slim AS runtime

ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PORT=8000 \
    WORKERS=4 \
    THREADS=2 \
    LOCATIONS_CSV=/app/data/locations.csv \
    LINKS_DIR=/app/links

RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser

WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY app.py engine.py gunicorn.conf.py ./
COPY data/ data/
COPY links/ links/

USER appuser
EXPOSE 8000

HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
    CMD ["python", "-c", "import os, urllib.request; urllib.request.urlopen(f'http://127.0.0.1:
{os.environ[\"PORT\"]}/health', timeout=2)"]

# Exec form: gunicorn runs as PID 1 and receives SIGTERM from `docker stop` directly.
# Bind address, workers and threads come from PORT / WORKERS / THREADS via gunicorn.conf.py.
CMD ["gunicorn", "app:app"]

# ---------- test: runtime + the venv that also has pytest + test suite ----------
FROM runtime AS test
COPY --from=testdeps /opt/venv /opt/venv
COPY tests/ tests/
```

```
# /app is root-owned and read-only for appuser, so pytest's cache plugin is switched off.
CMD ["python", "-m", "pytest", "-q", "-p", "no:cacheprovider"]
```

gateway/Dockerfile

```
# Gateway: nginx reverse proxy in front of the API + static file server for link files.
FROM nginx:1.28-alpine

# Read by the image's entrypoint, which runs envsubst on /etc/nginx/templates/*.template
# and writes the result to /etc/nginx/conf.d/ before nginx starts.
ENV API_HOST=api \
    API_PORT=8000 \
    MAX_UPLOAD=64m

RUN rm -f /etc/nginx/conf.d/default.conf \
    && mkdir -p /srv/links
COPY default.conf.template /etc/nginx/templates/default.conf.template

EXPOSE 80

HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retries=3 \
    CMD wget -qO /dev/null http://127.0.0.1/gateway-health || exit 1
```

gateway/default.conf.template

```
# ${API_HOST}, ${API_PORT} and ${MAX_UPLOAD} are filled in from the container's environment at start-up.

upstream proximity_api {
    server ${API_HOST}:${API_PORT};
    keepalive 16;
}

server {
    listen 80;
    server_name _;

    # The image links access.log to stdout (for `docker logs`); also keep a real
    # file so the gateway-logs volume holds a persistent copy.
    access_log /dev/stdout;
    access_log /var/log/nginx/gateway_access.log;

    # Link files are ~0.5 MB; the API itself rejects anything over 64 MB.
    client_max_body_size ${MAX_UPLOAD};

    location = /gateway-health {
        access_log off;
        default_type text/plain;
        return 200 "ok\n";
    }

    # Link files served over HTTP, so the API's "link=<URL>" mode can fetch them
    # (e.g. link=http://gateway/files/link.txt from inside the Docker network).
    location /files/ {
        alias /srv/links/;
        autoindex on;
        default_type text/plain;
    }

    location / {
        proxy_pass http://proximity_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
```

```
    }
}
```

docker-compose.yml

```
# Full stack:  docker compose up -d --build
#   gateway (nginx)  → published on http://localhost:${HOST_PORT:-8080}
#   api (gunicorn)   → reachable only inside the "proxnet" network as api:8000
# Settings come from .env (see .env.example); every one has a default here.

name: proximity-search

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
      target: runtime
    image: proximity-api:1.0
    restart: unless-stopped
    environment:
      PORT: 8000
      WORKERS: ${API_WORKERS:-4}
      THREADS: ${API_THREADS:-2}
      LOCATIONS_CSV: /app/data/locations.csv
      LINKS_DIR: /app/links
    volumes:
      - ./links:/app/links:ro
    expose:
      - "8000"
    networks:
      - proxnet

  gateway:
    build:
      context: ./gateway
    image: proximity-gateway:1.0
    restart: unless-stopped
    environment:
      API_HOST: api
      API_PORT: 8000
      MAX_UPLOAD: ${MAX_UPLOAD:-64m}
    ports:
      - "${HOST_PORT:-8080}:80"
    volumes:
      - ./links:/srv/links:ro
      - gateway-logs:/var/log/nginx
    depends_on:
      api:
        condition: service_healthy
    networks:
      - proxnet

networks:
  proxnet:
    driver: bridge

volumes:
  gateway-logs:
```

gunicorn.conf.py

```
"""gunicorn settings, read from the environment (loaded automatically from the working dir)."""

import os

bind = f"0.0.0.0:{os.environ.get('PORT', '8000')}"
workers = int(os.environ.get("WORKERS", 4))
```

```
threads = int(os.environ.get("THREADS", 2))
timeout = 120
accesslog = "-"
errorlog = "-"
```

`.dockerignore`

```
.git
.gitignore
.gitattributes
.venv
**/__pycache__
**/*.pyc
.pytest_cache
.env
report
tools
gateway
docker-compose.yml
Dockerfile
.dockerignore
README.md
start.sh
# loose copies of the assignment files in the repo root
/locations.csv
/link.txt
/Problem.txt
```

`.env.example`

```
# Copy to .env and edit; docker compose reads .env automatically.
HOST_PORT=8080
API_WORKERS=4
API_THREADS=2
MAX_UPLOAD=64m
```